

차량과 코드를 잇는 4년.

안드로이드 엔지니어 / 인포카 · OBD2 · 블루투스 · 안드로이드 오토

● 누적 다운로드

1,000만+

인포카 (B2C) 안드로이드 서비스

● 경력

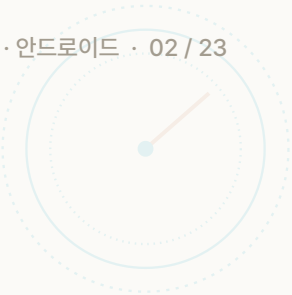
3년 10개월

인포카 안드로이드 · 2022.08 ~ 재직중

● 주요 프로젝트

5개

AI · OBD · 광고 · 오토 · 차계부



— 전체 차례 · TABLE OF CONTENTS

목차.

자기소개부터 5개 주요 프로젝트까지 — 차량 도메인에서 4년간 쌓은 엔지니어링 기록.

● 총 페이지

23p

● 소개 섹션

4 파트

● 주요 프로젝트

5개

● 도메인

차량 · OBD

소개		
03	자기소개	03
04	경력 사항	04
05	기술 보유	05
06	참여 프로젝트 목록	06
마무리		
23	감사 인사	23

주요 프로젝트		
P.01	AI 워크플로우 레거시 현대화	07-10
P.02	프리셋 대시보드 (OBD)	11-14
P.03	광고 미디어이션 전환	15-17
P.04	안드로이드 오토 플랫폼 확장	18-20
P.05	차계부 DB 리뉴얼	21-22

— 저는 이런 개발자입니다

자기소개.



빠르게 변하는 기술을 **실제 업무에 녹여내는 것**을 중요하게 생각하는 개발자입니다.

1,000만 다운로드 서비스의 사용자 접점부터 **운영·유지보수**까지 다루며,
최근엔 **AI로 저의 개발 방식**을 새롭게 다듬고 있습니다.

01

사용자 접점 기능

1,000만 다운로드 서비스의 블루투스 통신·푸시·알림 센터 등 일상 접점 기능 개발·운영.

02

레거시 현대화

MVC (자바)를 Kotlin·Hilt·Orbit MVI로 점진적·비파괴적으로 전환.

03

AI 워크플로우

기획→명세→구현→검증 풀 사이클을 AI와 함께 진행하는 방식을 본인 작업 영역에 적용.

04

운영·유지보수

릴리즈 모니터링·NPE 트러블슈팅·바인딩 리팩토링
·Play Console 정책 대응
·CS 분석까지.

05

투명한 협업

PM·디자이너·iOS와 스크럼, 기술 제약을 투명하게 공유·조율. iOS 크로스체크로 정책 통일.

소속

(주) 인포카
infoCar Inc.

직책

전임 연구원
주임 → 전임

기간

3년 10개월
2022.08 — 재직중

서비스

B2C · B2B
인포카 · 인포카 비즈

도메인

OBD2 · 블루투스
안드로이드 오토 · 광고

학력

신한대학교
컴퓨터공학과 졸업

— 한 회사 · 한 도메인

경력 사항.

2026.01 - 2026.05 · 진행 중

AI 워크플로우 레거시 아키텍처 현대화

폴 사이클 AI 협업 · MVC(Java) → Kotlin-MVI 점진 전환 · 14종 컨텍스트 문서 구조화

2025.09 - 2025.12

프리셋 대시보드 (OBD 차량 데이터)

B2B(인포카 비즈) 자산을 B2C에 이식 · 5종 엔진 × 3종 대시보드 · 자가 치유 메커니즘 + 복합키 마이그레이션

2025.03 - 2025.05

광고 미디어이션 전환

단일 AdMob → 5개 네트워크 워터폴 · Firebase Remote Config 9개 항목 동적 제어 · 광고 매출 ▲475%

2024.10 - 2025.02

안드로이드 오토 플랫폼 확장

Car App Library 신규 · 핸들러 양방향 동기화 · 포어그라운드 서비스로 앱 종료 후 단독 동작 · 11가지 안내 페이지

2024.06 - 2024.08

본앱 사용성 리뉴얼

차량 등록 방식 리뉴얼 · "운전→주행" 다국어 UX 통일 · 주유소·세차장·주차장 검색 · 영수증 API 리팩토링

2024.01 - 2024.05

제조사 데이터·Pro 구독 정책 / Biz 안정화

MOBD + 인포카 Pro 구독 정책 수정 · Biz 안드로이드 버그 수정 (로그인 실패 예외·동적 뷰 제약) · 자동 연결 실패 테스트

2023.11 - 2023.12

인포카 Biz Android 합류

본앱 + Biz 앱 동시 담당 시작 · Biz 지출 관리 디자인 수정 · CS 우선순위 처리 (대표이사 지정)

2023.07 - 2023.10

앱 핵심 기능 운영

블루투스 통신 · 푸시 · 알림 센터 · 정비 항목 UX/UI 2차 개발 · 차계부+정비 내부 테스트

2022.10 - 2023.06

차계부 시스템 리뉴얼

단일 테이블 → 정규화 다중 테이블 · 무손실 마이그레이션 · 주유소 API · 구글 맵 위치 기반 · 다단위 글로벌

2022.08 - 2022.09

입사 초기 · 터미널 프로그램

블루투스 connection · 소켓 통신 · UI · 로그 저장 (이메일·메모장). OBD 통신 기초 학습기.

— 다뤄온 도구와 언어

기술 보유.

현업 4년간 직접 다뤄온 도구·언어·아키텍처. 컬러 표기는 주력 기술.

언어와 아키텍처

01

언어 · 디자인 패턴 · 의존성 주입

Kotlin

Java

MVVM

Hilt

Coroutines / Flow

레포지토리 패턴

클린 아키텍처

UI와 디바이스

02

화면 · 디바이스 확장

Jetpack Compose

XML 레이아웃

Car App Library

반응형 레이아웃

데이터 시각화

머티리얼 3

데이터와 통신

03

저장소 · 통신 · 비동기

Room (SQLite)

DataStore

SharedPreferences

Firebase Remote Config

OBD2

Retrofit

SharedFlow / StateFlow

LiveData

AI 활용과 협업

04

AI 워크플로우 · 협업 도구

Claude Code CLI

ChatGPT CLI

명세 기반 개발 (SDD)

Git

슬랙

노션

피그마 (협업)

스크럼

광고 · 결제 · 미디어이션 · AdMob · AppLovin · ironSource · Mintegral · Pangle · Unity Ads · UMP (GDPR/CCPA)

PROJECT 03에서 5개 네트워크 워터폴 + 실시간 원격 제어 구현 경험

— 5개의 주요 프로젝트

참여 프로젝트.

최신순 5개 — 다음 페이지부터 프로젝트당 3~4장씩 상세 설명.

P.01 · 현재

2026.01 - 2026.05

**AI 워크플로우
레거시 현대화**

풀 사이클 AI 협업 ·
MVC(Java) → Kotlin-MVI
점진 전환

P.02

2025.09 - 2025.12

**프리셋 대시보드
(OBD 차량 데이터)**

B2B 자산의 B2C 이식 · 5종
엔진 × 3종 대시보드

P.03

2025.03 - 2025.05

**광고 미디어이션
전환**

단일 AdMob → 5개 네트워크
워터폴 · Remote Config

P.04

2024.10 - 2025.02

**안드로이드 오토
플랫폼 확장**

Car App Library 신규 · 양방
향 화면 동기화

P.05

2022.10 - 2023.06

**차계부 시스템
DB 리뉴얼**

단일 테이블 → 정규화 · 무손
실 마이그레이션

도메인 흐름**A**

차량 · 데이터 · 운전 환경

OBD2 프로토콜

차량 ECU 통신

블루투스 페어링

제조사 데이터

표준 PID

안드로이드 오토

기술 흐름**B**

언어 · 아키텍처 · 비동기

Kotlin · Java 혼용

Orbit MVI · MVVM

Hilt · Coroutines / Flow

Jetpack Compose

Room · DataStore

Firebase Remote Config

일관된 원칙**C**

매번 지킨 가치

레거시 점진 전환

라이프사이클 인식

단일 진실 공급원

반응형 레이아웃

무손실 마이그레이션

사용자 1,000만+

— PROJECT 01 · 진행 중

AI 워크플로우 레거시 현대화.

기획→명세→구현→검증 풀 사이클을 본인 작업 영역에 적용. 레거시 MVC(Java)는 보존하고, 신규 기능부터 Kotlin·MVI로 점진 전환.

기간	2026.01 ~ 2026.05 · 약 5개월 (진행 중)
인원	4명 (PM · 디자이너 · iOS · 안드로이드)
역할	본인 작업 영역 AI 협업 방식 정립, 안드로이드 레거시 현대화
기술	Kotlin, Java, Hilt, Coroutines/Flow, Orbit MVI, Claude Code CLI, 명세 기반 개발

핵심 이번 프로젝트의 의사결정

● 컨텍스트 문서

14종

AI 협업을 위한 구조화 문서

● 워크플로우 단계

5단계

답리서치 → PRD → SDD → 구현 → 검증

● 전환 방식

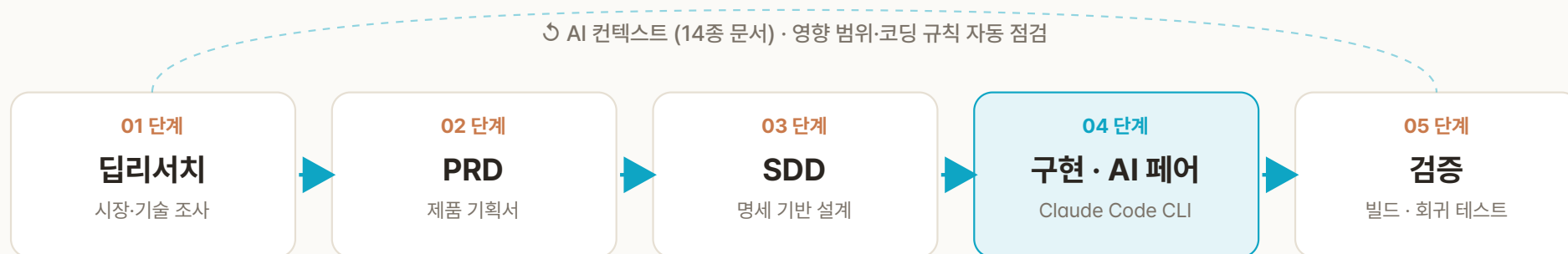
비파괴적

레거시 보존 · 신규 기능부터 전환

- 답리서치 → PRD → SDD → 구현 → 검증 풀 사이클을 본인 작업 영역에 적용
- 프로젝트 핵심 지식·코딩 규칙을 **14종 문서**로 정리해 AI 협업 컨텍스트 구축
- MVC(Java)를 기능 단위로 분리, 신규 기능부터 **Kotlin·MVI** 점진 전환
- 코드 수정 시 호출처·의존 관계를 AI가 추적해 **누락성 회귀·크래시 위험 사전 점검**

— 5단계 풀 사이클 협업

기획부터 검증까지.



각 단계의 산출물이 다음 단계 입력 — 단절 없는 컨텍스트 전달

산출물 각 단계별 결과물



A 풀 사이클 AI 협업

기획부터 검증까지 단계마다 AI가 14종 컨텍스트 문서를 참조해 함께 진행. 산출물이 다음 단계 입력이 되어 단절 없이 이어집니다.

B 영향 범위 자동 점검

코드 수정 시 호출처·의존 관계를 AI가 추적해 누락성 회귀 버그·크래시 위험을 사전 점검. 비파괴적 점진 전환 원칙으로 동작하는 레거시는 보존.

— 14종 컨텍스트 문서

AI 협업을 위한 지식 구조화.

프로젝트의 결정·규칙·아키텍처를 14종 문서로 정리. AI가 단계별로 필요한 문서만 참조하도록 슬라이스해서 활용.

기획·도메인

A

- 제품 개요 (product_overview)
- 도메인 용어집 (OBD2·ECU·BT)
- 사용자 페르소나

아키텍처

B

- 아키텍처 개요 (DataBridge·MVVM)
- 프로젝트 구조 (패키지 트리)
- 데이터 흐름 (Repository·Flow)
- 모듈 의존 관계

코딩 규칙

C

- 코딩 컨벤션 (Kotlin)
- 네이밍 규칙
- 에러 처리 가이드
- 로깅 가이드

레거시·전환

D

- 레거시 자산 목록 (MVC 인벤토리)
- 마이그레이션 플랜 (기능별 전환 순서)

검증·운영

E

- 테스트 전략 (회귀 케이스)
- 릴리즈 체크리스트 (빌드·심사)

설계 원칙 · 문서는 짧고 · 모듈화 · 인용 가능한 단위로. AI가 한 번에 너무 많은 컨텍스트를 받지 않도록, 단계마다 필요한 문서만 슬라이스해 제공.

명세 기반 개발(SDD) 원칙에서 차용

— 아키텍처 전환

비파괴적 점진 전환.

동작하는 레거시는 보존하고, 신규·수정 기능부터 새 아키텍처를 적용. 한 번에 다 뒤집기보다 안정적으로 둘이 공존하는 방식.

기존 (AS-IS)**MVC (Java)**

강결합 · 단일 구조

액티비티가 뷰·컨트롤러·일부 모델을 동시에 들고 있어 변경 시 영향 범위 추적이 어려움. 브로드캐스트 리시버 + 정적 싱글톤이 곳곳에서 상태 공유. 단위 테스트 작성 부담이 큼.

**목표 (TO-BE)****MVVM · Hilt · Orbit MVI**

관심사 분리 · 단방향 흐름

뷰모델이 UI 상태(StateFlow)와 사이드이펙트(SharedFlow)를 분리해 노출. DI는 Hilt로 일원화, 상태 전이는 Orbit MVI의 의도→상태 흐름으로 명확. 신규 기능부터 적용, 레거시는 어댑터로 연결.

전환 원칙**네 가지 원칙**

- **레거시 보존** — 잘 동작 중인 MVC 코드는 건드리지 않고 인터페이스로 격리
- **신규부터 적용** — 새 기능은 처음부터 Kotlin·MVI·Hilt로 작성
- **경계에서 어댑터** — 레거시 ↔ 신규 사이는 레포지토리·브릿지가 데이터 형식 변환
- **회귀 자동 점검** — AI가 호출처·의존 관계 추적해 영향 범위를 사전 보고

현 시점 성과**진행 중 결과 (2026.05 기준)**

- **14종 컨텍스트 문서 완성** — AI 페어 프로그래밍 기반 마련
- 본인 작업 신규 기능 **100% Kotlin·MVI** 적용 — 회귀 0건
- 코드 수정 1건당 영향 범위 점검 시간 **30~60초**로 단축 (기존 수동 검토 대비)
- 완료 항목과 진행 중 항목을 단계별로 검증해 가며 점진 확장 중 — 빅뱅 전환의 회귀 위험 회피

PROJECT 02

프리셋 대시보드 (OBD 차량 데이터).

B2B(인포카 비즈) 자산을 B2C(인포카)에 이식·확장하면서, 하드코딩된 센서 매핑을 사용자 선택형 + 엔진 타입별 5종 프리셋으로 재설계.

● 엔진 타입

5종

가솔린·디젤·LPG·전기·수소

● 대시보드

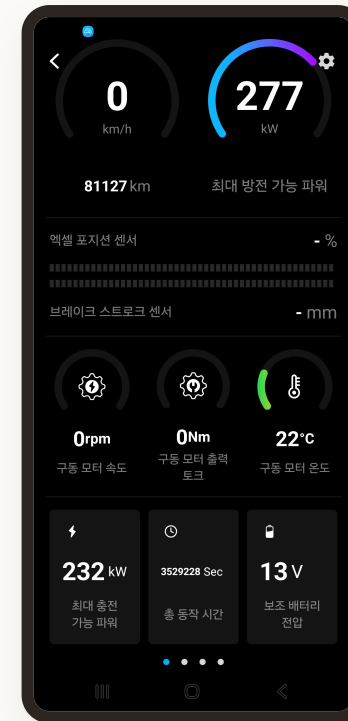
3종

주행 · TPMS · 배터리

● 레이아웃

4종

폰·태블릿 × 가로·세로



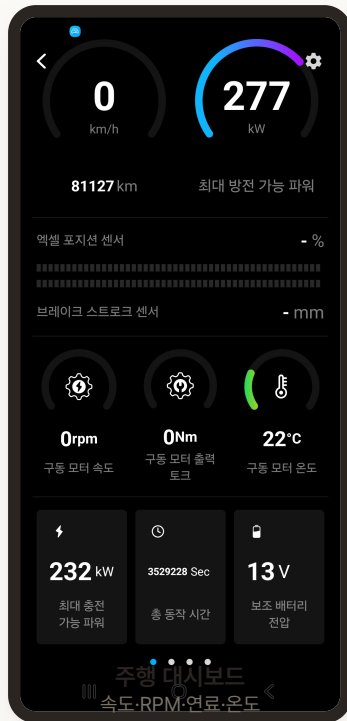
▲ 주행 대시보드 (실제 화면)

설계 방향 · 단순히 데이터 항목을 늘리기보다는 *엔진 타입별 인사이트 큐레이션*으로 접근. 디젤 → DPF·환경규제 / EV → 배터리 세부사항으로 사용자에게 의미 있는 데이터 묶음 제공.

기간	2025.09 ~ 2025.12 · 약 4개월
역할	정보 구조 설계 · 안드로이드 단독 구현 · B2B↔B2C 코드 통합
기술	Kotlin, Java, Jetpack Compose, DataStore, Room, MVVM, SharedFlow / LiveData

— 3종 대시보드 × 5종 엔진

엔진 타입별 인사이트 큐레이션.



A 사용자 선택형 매핑

한 의미값(예: "엔진 RPM")에 dataIndex가 여럿일 때 팝업에서 선택.

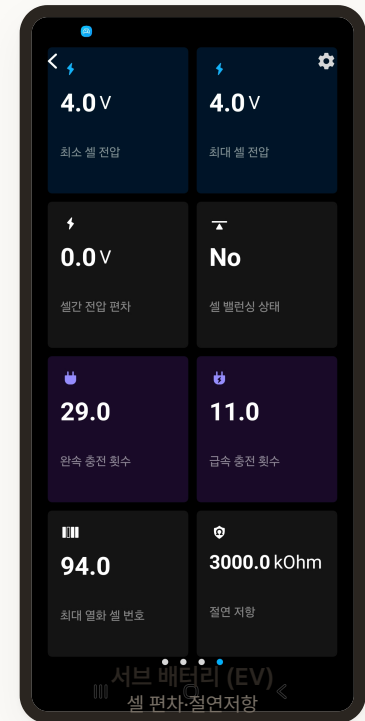
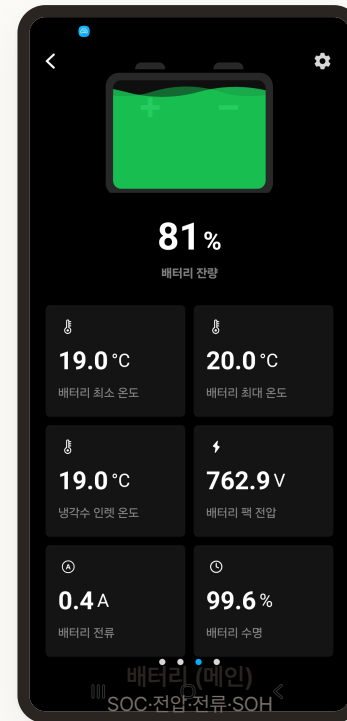
- RPM → `engine_rpm` / `standard_rpm`
- 모터 RPM → `mcu_motor_speed` / `rear` / `front`
- TPMS 위치 → 차량별 매핑 재설정



B 임계치 자동 환산

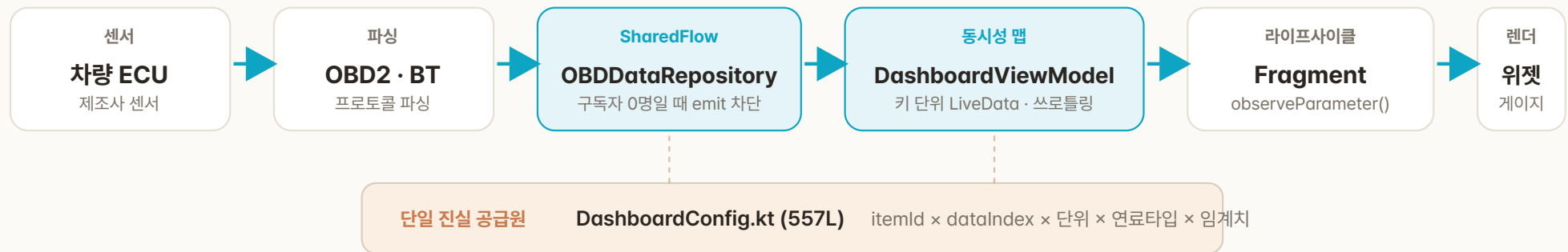
타이어 온도 80°C·압력 25psi 고정 → 사용자 설정 + 단위 자동 환산.

- 80°C ↔ 176°F (자동)
- 25 psi ↔ 1.72 bar (자동)
- 접미사 `_p` / `_t` 로 감지



— 데이터 파이프라인

ECU에서 위젯까지.

**SSOT** DashboardConfig — 단일 진실 공급원

itemId · dataIndexOptions · 기본 단위 · 페이지 · 연료타입 · 기본 임계치를 한 곳에 정의. 신규 항목 추가 시 1파일만 수정 — 기존엔 3~4파일 손대야 했음.

IF IDashboardDataStore — B2C·B2B 통일

공통 인터페이스(92줄) + 두 구현체. Fragment 코드는 BuildConfig 한 번만 검사 후 동일 API 사용. B2C·B2B 분기를 코드 깊은 곳에서 흡수.

핵심 클래스 레이어별 책임 분리

레이어	클래스	책임	규모
설정 · SSOT	DashboardConfig	모든 항목 정의 중앙화 ($\text{itemId} \times \text{fuelType} \times \text{unit} \times \text{threshold}$)	557 L
저장소 · 공통	IDashboardDataStore + 2 impl	인터페이스(92L) + B2C(248L) / B2B(539L) 두 구현	879 L
뷰모델	MOBDDashboardViewModel	ConcurrentHashMap · LiveData · 자가 치유 SharedFlow	237 L

— 핵심 처리와 정량 효과

8가지 기술 결정.

A · 자가 치유
키 invalid 자동 복구

제조사 DB 갱신 후 키가 무효해지면 SharedFlow로 알림 → 자동 재구독. 설정 손실 0건.

B · 복합키 마이그레이션
무단절 형식 전환

옛 data_index → 복합키(system+pid+rule) 자동 변환·저장. 앱 업데이트 시 사용자 설정 손실 0.

C · 임계치 단위 환산
psi↔bar · °C↔°F

_p / _t 접미사 감지 → 단위 변경 시 임계치도 자동 환산. 일관성 유지.

D · 표준 PID 폴백
제조사 미보유 대응

engine_rpm 실패 → standard_rpm 자동 시도. 제조사 데이터 미보유자도 가능 항목 표시.

E · 동시성 보장
ConcurrentHashMap

synchronized 없이 레이스 컨디션 방지 · computeIfAbsent로 DB 단일 조회 보장.

F · 동적 폰트 크기
Jetpack Compose

부모 너비 측정 → 단위·데이터 폰트 단계적 축소. 폰·태블릿 모두 잘리지 않음.

정량 효과 Before / After 비교

지표	Before	After	효과
바인딩 코드 길이	464줄	258줄	▼ 44%
신규 항목 추가 비용	3~4파일	1파일 (DashboardConfig)	▼ 70%+
지원 엔진 타입	2종 (가솔린·디젤)	5종 (+LPG·EV·수소)	▲ 2.5×
레이아웃 분기	1종	4종 (폰·태블릿 × 가로·세로)	▲ 4×
사용자 매핑 옵션	0 (하드코딩)	항목별 N:1 선택	신규
B2C·B2B 코드	Fragment 2벌	인터페이스 1벌 + 구현 2벌	통일

— PROJECT 03

광고 미디어이션 전환.

단일 AdMob → 5개 네트워크 워터폴 미디어이션. Firebase Remote Config 9개 항목으로 무배포 실시간 정책 제어.

기간	2025.03 ~ 2025.05 · 약 3개월
인원	4명 (PM · 디자이너 · iOS · 안드로이드)
역할	광고 미디어이션·원격 제어 시스템 설계·구현 (안드로이드)
기술	Kotlin, Java, AdMob, AppLovin · ironSource · Mintegral · Pangle · Unity Ads, Firebase Remote Config, UMP, CompletableFuture

배경 단일 AdMob의 한계 (30일 기준)

● 전면 광고 수익

\$869 /30일

405,778 노출 · eCPM \$2.14

● 네이티브 eCPM

\$1.42

전면 대비 ▼34% — 광고 피로도

● 활성 사용자 체류

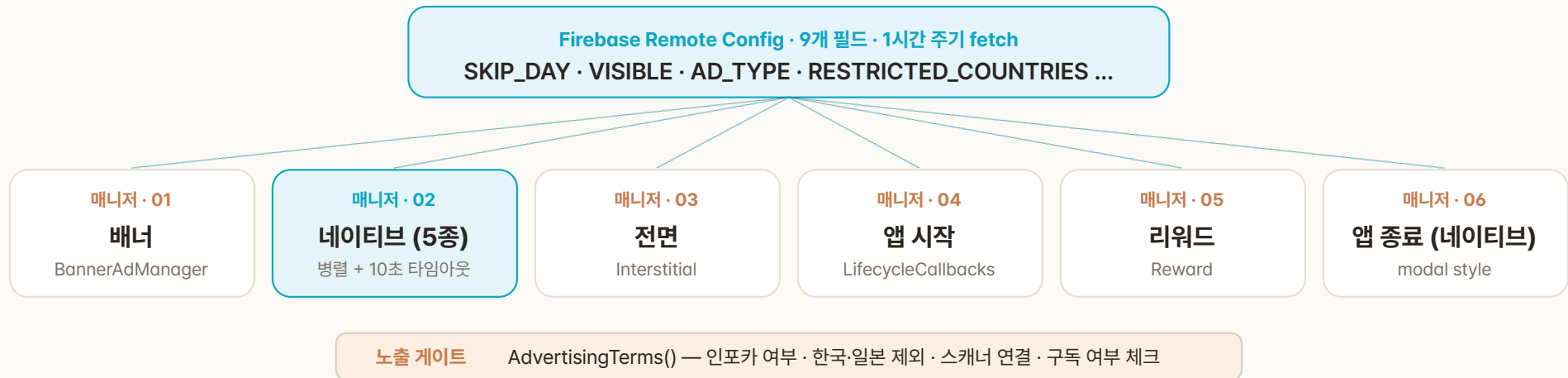
38 분

배너 광고 부재 → 지속 수익 0

- 국가별 노출 편중 — 브라질·멕시코 노출 많음, 고eCPM 미국·스위스·호주는 노출 부족
- 한국·일본 광고 부재 — 자체 광고만 운영, AdMob 수익 0
- 네이티브 팝업의 UX 저해 — CTR·eCPM 동반 하락. 자연스러운 배너 전환 필요

— 매니저 아키텍처 + 실시간 원격 제어

광고 타입별 매니저 + 무배포 정책.



A 광고 단위 ID 12개 세분화

위치별 광고 단위 분리 → CTR·eCPM 추적 가능.

- 네이티브 5종 (메인·사이드·바텀시트·자체배너·대시보드)
- 배너·전면·앱시작·리워드·앱종료

B Remote Config 9개 동적 제어

서버 배포 없이 광고 정책 변경.

- SKIP_DAY · DIALOG_VISIBLE · APP_OPENING
- REWARD_AD_SHOW · DASHBOARD_AD_TYPE
- RESTRICTED_COUNTRIES = ["KR", "JP"]

국가별 정책 자체 광고 + AdMob 통합 분기

한국 · 일본
자체 광고 우선

자체 배너 영역에 AdMob 네이티브 광고를 자연스럽게 섞어 노출.
표준 광고는 미노출.

해외 · 일반 사용자
네이티브 + 배너

메인·차계부·진단·기록 페이지에 네이티브 배너. 하단 텍스트형 배너 추가.

해외 · 타사 스캐너 연결
전체 광고 활성화

전면·앱시작·앱종료·팝업 + 목록 5개당 네이티브 광고 노출 (구독자 제외).

워터폴 + 정량 효과

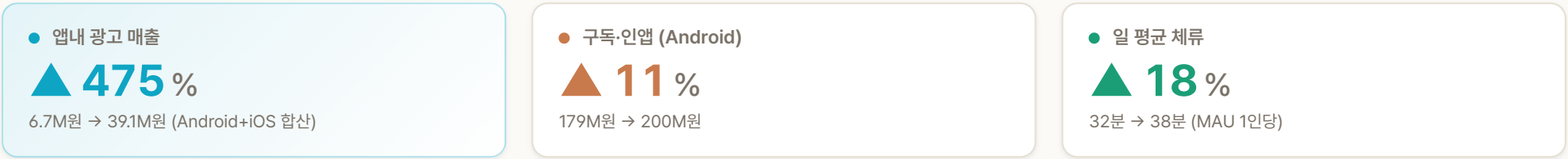
eCPM 워터폴 + 병렬 로딩 최적화.

워터폴 5개 네트워크 + AdMob 기반



사업 지표 2023 → 2024 매출 변화 (회사 전체 지표)

본인 기여 · 안드로이드 SDK 통합 · 매니저 구조 · UMP 자동화 · Remote Config 정책 제어 (매출 증가는 정책·사업 결정 등 복합 요인)



기술 효과 Before / After 시스템 변화

지표	Before	After	효과
미디어이션 네트워크	1개 (AdMob 단일)	5개 + AdMob 기반	▲ 5×
광고 단위 ID	~7개	12개 세분화	▲ 1.7×
정책 변경 속도	스토어 재배포	Remote Config 실시간	분 단 위
한국·일본 광고	없음 (자체광고만)	자체 + AdMob 통합	+a 수 익

PROJECT 04

안드로이드 오토 플랫폼 확장.

Car App Library로 신규 안드로이드 오토 앱 개발. 앱↔오토 양방향 동기화 + 앱 종료 후에도 오토 단독 동작하는 포어그라운드 서비스 구조.

● 화면

7 종

메인·연결·대시보드·표준
·Empty·데모

● 안내 페이지

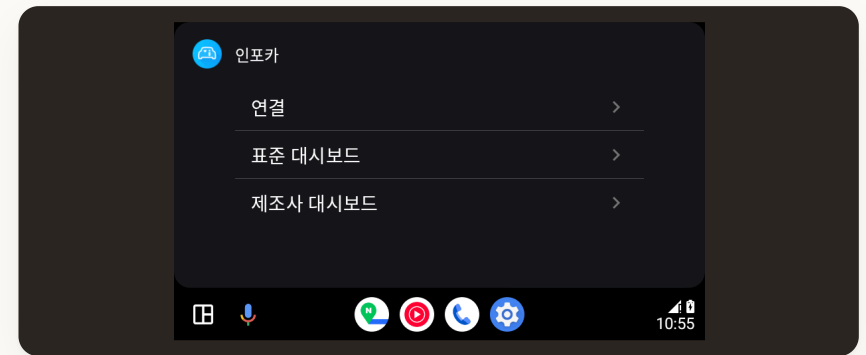
11 개

권한·연결·구매·다운로드
·검색

● OBD 매개변수

31 개

표준 매개변수 · 데모 시
물레이션



▲ 안드로이드 오토 메인 화면

정책 제약 인식 · 안드로이드 오토 / 카플레이는 운전 중 안전을 위해 *리스트·페인·메시지 템플릿* 3종만 허용. 그래픽 위젯·계기판은 사용 불가. → 정보 위계 압축 + 안내 페이지 11종으로 가이드 완성도 보강.

A 화면 구성 — 운전 안전 우선

- 최상위 메뉴 — 연결 · 표준 대시보드 · 제조사 대시보드 (구매 시 노출)
- 대시보드 페이지 — 앱 설정 항목과 동기화된 페이지 카운트
- 안내 페이지 — 권한·연결·구매·다운로드 등 11가지 상태별 가이드

B 차량 정보 표시 (연결 화면)

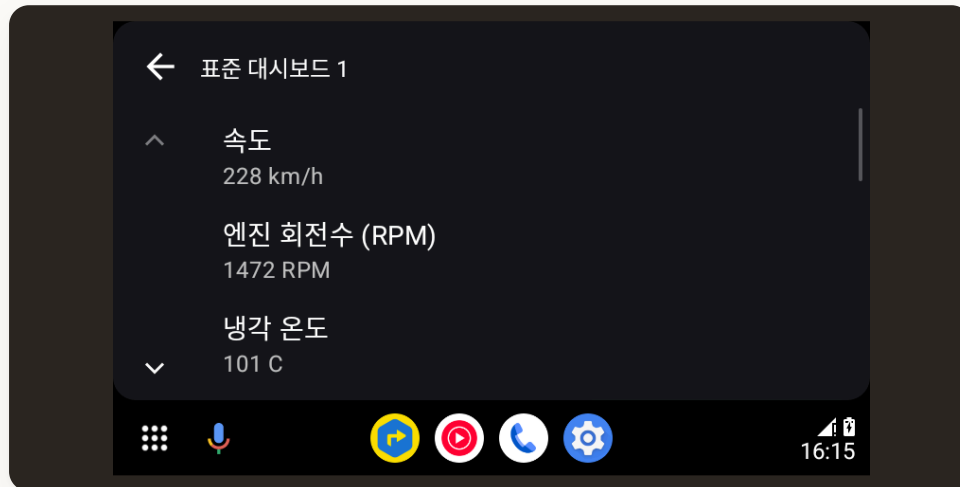
- | | |
|---------------|----------|
| 01 차량 이름 | 02 제조사 |
| 03 모델 | 04 연료 타입 |
| 05 연도 | 06 배기량 |
| 07 차대번호 (VIN) | |

— 화면 계층 + 양방향 동기화

앱과 오토를 잇는 핸들러.

화면 계층 7종 스크린 구성

- 메인 스크린 — 리스트 템플릿 (연결 · 표준 · 제조사)
- 연결 스크린 — 페인 템플릿 (OBD/ECU 상태 + 차량정보 7필드)
- 대시보드 스크린 — 리스트 템플릿 (페이지 동적 카운트)
- 표준 데이터 스크린 — 페인 템플릿 (실시간 매개변수)
- Empty 스크린 — 메시지 템플릿 (11가지 상태)
- 데모 화면 2종 — 스캐너 없이 시연 가능

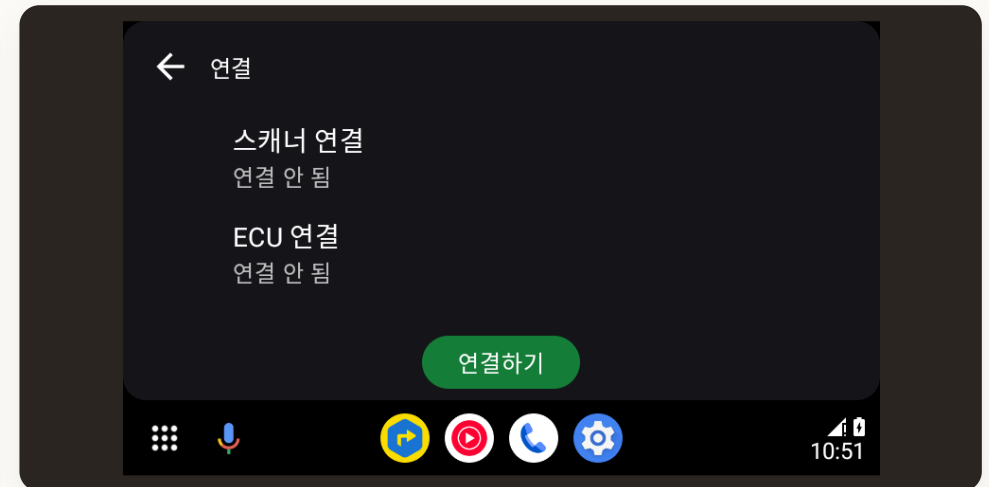


▲ 표준 대시보드 화면

양방향 핸들러 메시지 패싱

앱과 오토 사이를 *핸들러 메시지*로 동기화. 메인 루퍼 기반으로 ANR 방지.

- **what = 0** → 연결 스크린 갱신
- **what = 1** → 대시보드 스크린 갱신
- **what = 2** → 표준 데이터 스크린 갱신
- **전역 브릿지** — Status_DataBridge로 CarContext·상태 공유
- **독립 동작** — 앱 종료 후에도 Auto만 단독 실행



▲ 연결 화면 (스캐너·차량 상태)

— 상태 머신 + 온보딩

상태 코드 + 11개 안내 페이지.

A 연결 상태 머신 (0-3)

OBD·ECU 두 채널의 상태를 0-3 코드로 명확히. 둘 다 2일 때만 데이터 표시.

코드 0 미연결

→ Empty 스크린

코드 1 연결 중

→ 스피너 표시

코드 2 연결 완료

→ 데이터 표시

코드 3 실패

→ 오류 안내

B 11개 안내 페이지 (Empty)

상황별 구체적인 가이드로 첫 사용 경험 보호.

- 오토 권한 미설정 · 위치 권한 필요
- 오버레이 권한 필요
- 스캐너 먼저 연결 필요
- 제조사 데이터 구매 필요
- 제조사 프로파일 다운로드 필요
- 제어유닛(ECU) 검색 필요
- 제조사 설정 진행 중 · 진단 설정
- 대시보드 항목 비어 있음 · 일반 오류

포어그라운드 앱 종료 후 단독 동작

- **FOREGROUND_SERVICE_TYPE_LOCATION** — 위치 권한이 있을 때만 사용, 없으면 폴백 경로
- **SecurityException** 처리 — API 33+ 권한 제약을 우아하게 회피
- **CarContext** 유지 — 메인 앱 종료해도 오토는 계속 데이터 갱신 (배터리 영향 최소화)
- **재진입 시 복원** — 기존 CarContext·대시보드 상태 자동 복원, 사용자 작업 손실 0

— PROJECT 05

차계부 시스템 DB 리뉴얼.

단일 테이블 → 정규화 다중 테이블로 DB 재설계. 1,000만 사용자 데이터를 무손실 마이그레이션. 구글 맵 위치 기반 + 글로벌 다단위 대응.

배경 왜 리뉴얼했는가

- 단일 테이블의 한계 — car_log 한 테이블에 차량·주유·정비·소모품이 혼재 → 중복·확장 한계
- 무손실 요구 — 1,000만 다운로드 서비스의 누적 사용자 데이터를 절대 손실 없이 변환
- 위치 기반 확장 — 주유소 API 적용 + 구글 맵 장소 검색 통합
- 데이터 복원 안전망 — 복구 API 작업 + 마이그레이션 로직 수정으로 사용자 무손실 보장
- 글로벌 단위 대응 — 거리(km/mile), 연비(km/L, MPG), 통화 등 지역별 단위 시스템

● 기간

9

개월

설계 · 마이그 · 검증 풀 사이클

● 테이블 수

4

개

vehicle · fuel · maintenance · consumable

● 대상 사용자

1,000

만+

누적 다운로드 · 데이터 손실 0

기간	2022.10 ~ 2023.06 · 약 9개월
인원	4명 (PM · 디자이너 · iOS · 안드로이드)
역할	DB 재설계·마이그레이션 · 위치 기반 기능 (안드로이드)
기술	Kotlin, Java, Room (SQLite), 구글 맵 API, 안드로이드 XML

— 스키마 재설계 + 위치·다단위 확장

단일 → 4개 테이블 · 글로벌 대응.

기존 (AS-IS)

car_log (단일 테이블)

차량 + 주유 + 정비 + 소모품 혼재

type 컬럼으로 종류 구분. 중복 컬럼 다수·NULL 비율 높음. 신규 종류 추가 시 ALTER TABLE 필요.



목표 (TO-BE)

vehicle · fuel · maintenance · consumable

정규화 4테이블 · 외래키 관계

vehicle을 마스터로, 나머지는 vehicle_id 외래키로 참조. 중복 제거, 인덱스 효율 ↑, 신규 종류는 새 테이블만 추가.

전략 무손실 마이그레이션

- 구 스키마 백업 → 신 스키마 빈 테이블 생성 → 변환 INSERT
- 실패 시 트랜잭션 롤백, 백업으로 즉시 복구
- 검증: ROW COUNT 비교 + 샘플링 데이터 무결성
- 점진 배포 — 일부 사용자 먼저 적용 후 전체 확대

확장 구글 맵 + 다단위 글로벌

- 주유소·정비소 장소 검색 + 거리순 정렬
- 거리: km · mile · yard · foot
- 연비: km/L · mile/gal (MPG), 볼륨 L · gal
- 통화·날짜 — 디바이스 로케일 기반 자동

● 구조 전환

1 → 4 테이블

정규화 다중 테이블 · 관심사 분리

● 데이터 손실

0 건

무손실 마이그레이션 검증 완료

— 마지막 페이지

읽어주셔서
감사합니다.

연락처

010-9376-2966

이메일

dkwkrhrh07@naver.com

깃허브

github.com/AIMA0719

